

28. Principy modelování a simulace systémů (systémy, modely, simulace, algoritmy řízení simulace)

28. Principy modelování a simulace systémů

SZZ okruh 28 (FIT BUT). Od reálného systému přes model k simulaci.

Shrnutí

Systémy a modely

- **Systém** = prvky + vazby; formálně $S = (U, R)$. Dělení: spojitý/diskrétní/kombinovaný, deterministický/nedeterministický.
- Vztahy: **izomorfní** (bijekce), **homomorfní** (surjekce — princip modelování).
- **Model** = napodobenina systému; **modelování** = experiment $\rightarrow$ znalosti $\rightarrow$ abstraktní model $\rightarrow$ simulační model $\rightarrow$ **verifikace** (AM $\rightarrow$ SM) a **validace** (model $\rightarrow$ realita) $\rightarrow$ simulace.
- Klasifikace modelů: konceptuální, deklarativní (automaty, Petriho sítě), funkcionální, popsané rovnicemi, prostorové, multimodely.

Simulace

- **Simulace** = získávání znalostí experimentováním s modelem (cena, rychlost, bezpečnost; problém validity a výpočetní náročnosti).
- Typy: spojitá / diskrétní / kombinovaná; **Petriho sítě**, **Monte Carlo** (stochastická — integrály, osvětlení), **celulární automaty** (Game of Life), Markovovy modely.
- Čas: **reálný** / **modelový** / **strojový**.

Algoritmy řízení simulace

- **Next-event** (diskrétní) — **kalendář událostí** (seřazené aktivační záznamy), zpracování po událostech.
- **Spojitá** — funkce Dynamic (vstupy integrátorů) + numerická metoda ([okruhy/24-numerické-metody | Euler/RK]) + hlavní cyklus s krokem a **dokročováním**.
- **Kombinovaná** — kombinace + detekce **stavových událostí** (půlení intervalu na přesný okamžik).

Numerické metody viz [[okruhy/24-numerické-metody]]; Monte Carlo / náhodnost viz [[okruhy/31-pravdepodobnost-statistika]].

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Základní pojmy $\rightarrow$ [[#Systémy a modely]] - *Systém vs. model vs. simulace?* $\rightarrow$ Systém = část reality; model = jeho napodobenina; simulace = experimentování s modelem. - *Verifikace vs. validace?* $\rightarrow$ Verifikace = shoda abstraktního a simulačního modelu; validace = shoda modelu s realitou.

Typy simulace □ [[#Simulace]] - *Diskrétní / spojitá / kombinovaná?* □ Dle povahy prvků modelu (skokové změny × spojitě × obojí).

Algoritmy řízení □ [[#Algoritmy řízení simulace]] - *Kalendář událostí?* □ Seřazená struktura aktivních záznamů (čas, priorita, událost); next-event bere první. - *Dokročování?* □ Zkrácení/prodloužení posledního kroku tak, aby čas přesně odpovídal konci simulace / okamžiku stavové události.

Stochastické metody □ [[#Simulace]] - *Monte Carlo?* □ Náhodné experimenty + pravděpodobnost/průměr □ výsledek (integrály, obsah/objem); přesnost roste s počtem experimentů.

Plné znění (ke studiu)

Systémy

Systém je soubor elementárních částí (prvků systémů), které mají mezi sebou určité vazby. Můžeme je dělit na:

- **Reálné systémy**
- **Nereálné systémy** - Fiktivní, ještě neexistující.
- **Spojitě** - Všechny prvky mají **spojité chování**.
- **Diskrétní** - Všechny prvky mají **diskrétní chování**.
- **Kombinované** - Obsahuje **spojité i diskrétní** prvky.
- **Deterministické** - Všechny prvky jsou **deterministické**.
- **Nedeterministické** - Alespoň jeden prvek s **nedeterministickým** chováním.

Formálně se jedná o dvojici **S = (U, R)**, kde:

- **U = {u1, u2, ..., un}** je univerzum - konečná **množina prvků** systému. Každý prvek je navíc tvořen dvojicí (X, Y), **u = (X, Y)**, kde:
 - **X** je množina všech **vstupních proměnných**.
 - **Y** je množina všech **výstupních proměnných**.
- **R** - Množina všech **propojení** (relací), která je podmnožinou **kartézských součinů vstupů a výstupů** jednotlivých prvků.
Propojení prvku u_i s u_j :

[[media/szz-28/media/image15.png]]

[[media/szz-28/media/image2.png]]

Příklad formálně definovaného systému může vypadat následovně: [[media/szz-28/media/image3.png]]

Vazba mezi prvky může být **sériová, paralelní** a nebo **zpětná**.

[[media/szz-28/media/image19.png]]

Vztahy mezi systémy

- **Izomorfní systémy** - Systémy **S1 = (U1, R1)** a **S2 = (U2, R2)** jsou izomorfní pokud mezi nimi **existuje bijekce**:

1. Prvky **U1** lze **vzájemně jednoznačně přiřadit U2 (bijektivní zobrazení - 1:1)**.

2. Prvky **R1** lze **bijektivně** zobrazit na **R2** se stejně **orientovanými** vztahy na prvky univerz.

[[media/szz-28/media/image21.png]]

- **Homomorfní systémy** - Je základním **principem modelování**, systémy jsou homomorfní, pokud mezi nimi **existuje surjekce**:

1. Prvkům **U1** je možné přiřadit **jednoznačně** prvky **U2** (**surjektivní zobrazení** - $N:1$).
2. Prvkům **R1** je možné jednoznačně přiřadit prvky **R2** se **stejně orientovanými vztahy** s univerzy. [[media/szz-28/media/image22.png]]

Chování systémů

Každému časovému průběhu **vstupních proměnných přiřazuje časový průběh výstupních proměnných**. Je dáno vzájemnými interakcemi mezi prvky. Jinak řečeno jedná se **způsob, jakým systém převádí vstupy na výstupy**. Jedná se o zobrazení:

[[media/szz-28/media/image6.png]]

Ekvivalence chování systémů - Pokud **stejně podněty** u obou systémů vyvolají **stejně reakce**, systémy mají ekvivalentní chování.

Modely

Model je **napodobenina** systému jiným systémem. Reprezentuje znalostí, které máme o systému. Klasifikace:

- fyzikální modely, matematické modely - přírodní zákony jsou matematické modely (Ohmův zákon: $U = R \cdot I$).

Modelování

Proces vytváření modelů systému na základě znalostí, které o daných systémech máme. Proces modelování je následující:

1. Experimenty a pozorování reality (někdy není možné, modelujeme a simulujeme neexistující věc),
2. Získ znalostí o modelovaném systému,
3. Tvorba **abstraktního modelu** - formování zjednodušeného popisu systému,
4. Tvorba **simulačního modelu** - zápis abstraktního modelu programem,
5. **Verifikace a validace** - Ověřování správnosti modelu.
6. **Simulace** - experimentování se simulačním modelem, [[media/szz-28/media/image18.png]]
7. Analýza a interpretace získaných výsledků, což vede zpět na bod 2 a celý proces lze opakovat např. dokud nejsme spokojeni s výsledkem.

Verifikace a validace modelů

- **Verifikace modelu** - Ověřujeme **korespondenci abstraktního a simulačního modelu**, tj. izomorfní vztah mezi AM a SM. Předchází vlastní etapě simulace.
- **Validace modelu** - Snažíme se dokázat, že **skutečně pracujeme s modelem adekvátním modelovanému systému (reálnému systému, o kterém chceme zjistit více informací)**. Velmi obtížné a nelze absolutně dokázat. Pokud chování modelu neodpovídá předpokládanému

chování, musí se model modifikovat nebo je třeba nalézt příčinu odchylky. Je nutné neustále porovnávat informace, které o modelu máme a které získáváme simulací.

Klasifikace modelů

- **Konceptuální** - Jejich komponenty **nebyly** (zatím) **přesně popsány** ve smyslu teorie systémů. Obvykle se používají v **počáteční fázi** modelování pro ujasnění souvislostí a **komunikaci v týmu**. Mají formu **textu** nebo **obrázku**.

[[media/szz-28/media/image23.png]]

- **Deklarativní** - Popis **přechodů** mezi **stavy** systému. Model je definován **stavy** a **událostmi**, které **způsobí přechod** z jednoho do druhého za jistých podmínek. Vhodné především pro **diskrétní modely**. Obvykle zapouzdřeny do objektů (**konečné automaty**, **petriho sítě**, **událostmi řízené systémy s kalendářem**, **Markovovy modely**, ...).

[[media/szz-28/media/image27.png]]

- **Funkcionální** - Grafy zobrazující **funkce a proměnné**. Buď je uzel grafu proměnná nebo funkce (**systémy hromadné obsluhy**, **bloková schémata systémové dynamiky**).

[[media/szz-28/media/image9.png]]

- **Popsané rovnicemi (constraint)** - Rovnice (**algebraické, diferenciální, diferenční**), jedná se např. o rovnice kyvadla, RC článku, [[media/szz-28/media/image5.png]]
- **Prostorové (spatial)** - Rozdělují systém na prostorově menší ohraničené podsystemy (**Parciální diferenciální rovnice**, **celulární automaty**, L-systémy, N-body problém).
- **Multimodely** - Je složen z modelů různého typu, které jsou obvykle heterogenní (spojité + diskrétní, spojité + fuzzy, HLA).

Simulace

Proces získávání **nových znalostí** o systému pomocí **experimentování s jeho modelem**.

Výhody

- cena,
- rychlost,
- bezpečnost,
- někdy jediný způsob (srážky galaxií),
- ...

Problémy

- kontrola validity (nemusí být validní a nepoznáme to),
- náročnost na vytváření,
- výpočetní náročnost,
- nepřesnost numerického měření,
- problémy stability numerických metod,

- ...

Postup Opakované řešení modelu, experimentování s ním. Opakuje se, dokud nezískáme **dostatek informací** o chování nebo dokud **nenajdeme parametry** pro které má systém **žádané chování**. Jeden simulační cyklus vypadá následovně:

1. Nastavení hodnot parametrů a počátečního stavu modelu.
2. Zadání vstupních podnětů z okolí při simulaci.
3. Vyhodnocení výstupních dat (informace o chování systému).

Typy simulace

- **Podle popisu modelu:**
 - **Spojitá/diskrétní/kombinovaná**
 - Kvalitativní/kvantitativní
- **Podle simulátoru**
 - Na **analogovém/číslicovém** počítači, fyzikální
 - **Real-Time** simulace
 - **Paralelní a distribuovaná** simulace

Analytické řešení modelů

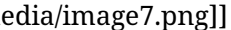
Popis chování modelu matematickými vztahy a jeho matematické řešení. Vhodné pro jednoduché systémy nebo zjednodušený popis složitých. Dosazením korektních hodnot získáme řešení (např. model volného pádu ve vakuu - v atmosféře už by byl ale složitější).

Čas

- **Reálný** - Ve kterém probíhá skutečný děj.
- **Modelový** - Časová osa modelu (nemusí být synchronní s reálným) - čas, podle kterého se řídí simulace.
- **Strojový** - Čas CPU spotřebovaný na výpočtu programu.

Časová množina

Množina všech časových okamžiků, ve kterých jsou definovány hodnoty vstupních, stavových a výstupních proměnných prvků systému

- **Diskrétní** - {1,2,3,4,5}.
- **Spojitá** - <1.0, 5.0>. Tato se na počítači diskretizuje. 



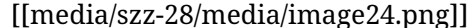
Simulační metody

Petriho síť

Petriho síť je formálně definována jako pětice $N = (P, T, F, W, C, M_0)$, kde:

- $P = \{ p_1, p_2, \dots, p_m \}$ je konečná množina míst,

- $T = \{ t_1, t_2, \dots, t_n \}$ je konečná množina přechodů, $P \cap T \neq \emptyset$ a $P \cap T = \emptyset$,
- $F \subseteq (P \times T) \cup (T \times P)$ je incidenční relace definující vstupní funkci, která definuje orientované křivky z míst do přechodů, a výstupní funkci, která definuje orientované křivky z přechodů do míst,
- váhová funkce $W: F \rightarrow \{1, 2, \dots\}$,
- kapacity míst $C: P \rightarrow \mathbb{N}$,
- $M_0: P \rightarrow \mathbb{N}$ je počáteční značení.

Značení znamená přiřazení žetonů do míst Petriho sítě. Orientovaná křivka směřující z místa **pi** do přechodu **ti** definuje **pi** jako **vstupní místo přechodu ti**. Orientovaná křivka směřující z **přechodu ti** do **místa pj** pak definuje **pi** jako **výstupní místo přechodu ti**. Vstupní místo je místo, ze kterého je žeton při provádění přechodu **odebrán**, a naopak výstupní místo je místo, do kterého je **žeton** po provedení přechodu **přesunut**. Graficky jsou **místa** v Petriho sítích značena elipsami, obvykle je ale znázorňujeme **kružnicemi**. **Přechody** jsou značeny **obdélníky**, orientované **křivky šipkami**. **Žetony** mohou být značeny **tečkami** nebo **číslem**, které vyjadřuje počet teček – žetonů. Značení žetonů se obvykle wpisuje do značky místa. Příklad systém M/M/1 dle Kendalovy klasifikace: 

Metoda Monte Carlo

Experimentální numerická (simulační) metoda. Řeší úlohu experimentování se stochastickým (pracující s náhodností, pravděpodobností: náhodnou proměnnou) modelem. Využívá vzájemného vztahu mezi hledanými veličinami a pravděpodobností, se kterými nastanou jevy. Vyžaduje generování náhodných čísel. Není příliš přesná. Vhodné, když jsou běžné numerické metody nepraktické. Jednoduchá implementace (existuje více variant).

1. Vytvoříme stochastický model.
2. Provádíme náhodné experimenty.
3. Získanou pravděpodobnost nebo průměr použijeme pro výpočet výsledku.

Použití:

- výpočet obsahu, objemu těles (nemusíme znát obor hodnot funkce)
- výpočet integrálů, zejména vícerozměrných,
- řešení diferenciálních rovnic,
- výpočet osvětlení scény (path/ray tracing),
- provádění náhodných procházek při trénování algoritmů posilovaného učení.

Přesnost (N je počet provedených experimentů): 

Celulární automaty

Celulární automaty jsou diskrétní systémy. Je možné je implementovat jako pole, vyhledávací tabulku (nenulové buňky). Existují také reverzibilní automaty, u kterých je možné se vracet nazpět v simulaci. Celulární automaty jsou tvořeny:

- **Buňka (cell)** - **základní element**, může být v jednom z **konečného počtu stavů**, např. $\{0, 1\}$.
- **Pole buněk (lattice)** - **N-rozměrné** pole (obvykle 1D nebo 2D). Rovnoměrné rozdělení prostoru, může být **konečné nebo nekonečné**.

[[media/szz-28/media/image25.png]]

[[media/szz-28/media/image13.png]]

- **Okolí (neighbourhood)** - Typy se liší počtem a pozicí **okolních buněk**.
- **Pravidla (Rules)** - Funkce stavu buňky a jejího okolí definující nový stav buňky v čase: **$s(t + 1) = f(s(t), Ns(t))$** .

Lze je rozdělit do **4 tříd**:

- **Třída 1** - Po konečném počtu kroků dosáhnou jednoho ustáleného konkrétního stavu.
- **Třída 2** - Dosáhnou periodického opakování nebo zůstanou stabilní.
- **Třída 3** - Chaotické chování (fraktální útvary).
- **Třída 4** - Kombinace běžného a chaotického chování (např. life) - nejsou reverzibilní.

Nejnámější celulární automat je hra **Life** (Game of Life).

Markovovy modely

Jedná se o Markovské procesy, které splňují Markovovu vlastnost, tj. následující stav závisí pouze na aktuálním stavu (nezávisí na minulosti). Používají se pro analytické řešení systémů hromadné obsluhy typu M/M/x. [[media/szz-28/media/image10.png]]

Metoda snižování řádu derivace

1. Osamostatnit nejvyšší řád derivace.
2. Zapojit všechny integrátory za sebe a na vstupu prvního zapojit (modifikovaný - násobení, přičítání) výsledek
3. Tato metoda funguje, pokud nejsou derivované vstupy derivace vstupů (x' , x'' , ...).

Metoda postupné integrace

[[media/szz-28/media/image26.png]]

[[media/szz-28/media/image17.png]]

1. **Osamostatnit nejvyšší řád** derivace.
2. Postupná integrace rovnice a zavádění nových stavových podmínek.
3. Výpočet nových počátečních podmínek.
4. Podmínka: konstantní koeficienty. [[media/szz-28/media/image16.png]]

Algoritmy řízení simulace

[[media/szz-28/media/image12.png]]

Různé algoritmy pro diskrétní, spojitý a kombinované simulace.

Next event (řízení diskretní simulace)

Jedná se algoritmus řízení diskretní simulace. Vypadá následovně:

1. **Inicializace** času, kalendáře, modelu.
2. Dokud **není kalendář prázdný**, vyjmi první záznam z kalendáře (záznamy jsou seřazeny podle času vzestupně).
3. Pokud aktivační čas události **přesáhl** čas konce simulace, ukonči simulaci.
4. **Nastav** čas na aktivační čas události.
5. Proveď **chování**, které popisuje událost a **vrať se** na bod 2.

Kalendář

Uspořádaná **datová struktura** uchovávající aktivační záznamy budoucích událostí. Každá naplánovaná událost má v kalendáři záznam, který je tvořen minimálně:

- **aktivačním časem**,
- **prioritou**,
- **vykonávanou událostí** (odkaz na funkci k vykonání).

Kalendář umožňuje **výběr prvního záznamu** s nejmenším aktivačním časem a **vkládání/rušení záznamu**.

Řízení spojitě simulace

Spojité simulace je složena ze tří částí:

- metoda/funkce **Dynamic**, ve které dochází k **aktualizaci vstupů integrátorů**,
- **numerická metoda** (Euler, RK) ve které se **pro každý integrátor počítají nové stavy**,
- **hlavní cyklus**, který řídí simulaci (volá předchozí metody), **inkrementuje čas dle kroku** a sleduje, jestli nebylo dosaženo konce simulace a případně provádí **dokročení**.

Dokročení může být řešeno dvěma způsoby:

- pokud do konce simulace zbývá **menší časový okamžik**, než je nastavený krok, lze poslední **krok zkrátit** tak, aby čas po jeho provedení přesně odpovídal konci simulace. (příliš malý krok může způsobit nepřesnost)
- pokud do konce simulace **zbývá krok a kousek**, můžeme poslední **krok prodloužit** tak, aby čas po jeho provedení přesně odpovídal konci simulace.

Příklad řízení spojitě simulace bez dokročení:

Řízení kombinované simulace

[[media/szz-28/media/image14.png]]

Je to kombinace spojitě a diskretní simulace. U kombinované simulace je nutné řešit kombinaci **stavových událostí** a **numerické integrace**. Je nutné **detekovat změnu** stavových podmínek a **přesně dokročit** na čas, kdy ke změně dochází (problém příliš malého kroku). Ke **stavovým událostem** dochází při **změnách stavových podmínek**, nelze je naplánovat. Změnu stavové podmínky může být **obtížné detekovat** z důvodu **nepřesnosti numerického výpočtu** nebo **příliš dlouhého kroku**. [[media/szz-28/media/image11.png]]

Algoritmus řízení kombinované simulace pracuje následovně:

1. **Inicializace** stavu, času, modelu atd.
2. **Kontrola**, že není dosažen čas **konce simulace** a její případné ukončení.
3. **Uložení** aktuálního stavu a času.
4. Provedení jednoho **kroku numerické integrace**.
5. Kontrola, jestli nedošlo ke změně stavových podmínek. Pokud ne, pokračuje se bodem 2, jinak bod 6.
6. Hledání **okamžiku** změny **stavové podmínky** (využití uložených hodnot v **kroce 3**) např. metodou půlení intervalů. Okamžik se hledá **s přesností minimálního kroku** (menší krok by vedl na nepřesnost, stejně jako na nepřesnost vede nepřesné určení času změny stavové podmínky)

Pseudokód algoritmu:

[[media/szz-28/media/image8.png]]

koncový_čas - čas konce simulace **nebo** čas další naplánované události v kalendáři (podle toho co nastane dříve)

Kombinovaná simulace může obsahovat i diskrétní události a kombinuje algoritmus next event a algoritmus pro spojitou simulaci. Nejdříve se provedou všechny události naplánované pro daný čas. Potom se spustí spojitá simulace a jede se ta smyčka co je výše v algoritmu (během spojitě simulace se může posunout koncový čas, pokud se v důsledku stavové události naplánuje další událost). Po dosažení času další události v kalendáři se vrátí na začátek a pokračuje se opět vyhodnocením všech událostí v aktuálním čase (next event), pak zas spojitá část simulace...

Kromě té spojitě simulace a řešení stavových podmínek popsanych výše se tam řeší i kalendář událostí - tzn. na časové ose přesně víme, kdy se ta událost z toho kalendáře má vykonat a proto se tam v případě toho typu událostí nemusí řešit dokročení, ale numerická integrace přímo ví, jak velký krok má udělat.

Řízení simulace číslicových obvodů

Tato simulace je řízená událostmi a ukládání **velkého množství** událostí do kalendáře je **nepraktické/problematické**. Používá se proto princip **selektivního sledování**, kdy dochází k vyhodnocování pouze těch prvků, na které má vliv změna na vstupu. Používá se např. **pevný krok** pro změnu času. Problematické mohou být zpětné vazby v obvodech a nastavení počátečních hodnot signálů.

Princip algoritmu:

1. **Inicializace** modelu, plánování, ...
2. Dokud je naplánovaná událost, tak pokračuj s dalším bodem, jinak konec simulace.
3. Nastav hodnotu modelového času na **T** (pevným krokem, na základě první naplánované události, ...)
4. Pro všechny události, které jsou **naplánované** na tento čas **T** proved:
 1. **odeber** událost z plánovaných události (kalendáře),
 2. **aktualizuj** hodnoty signálů,
 3. všechny připojené prvky na tyto signály **zařaď do množiny M**.
5. Projdi všechny prvky v množině **M** a jestli změna na jeho vstupu způsobí **změnu jeho výstupu**, **naplánuj** jeho obsluhu jako novou událost.
6. Pokračuj bodem 2.

Pseudokód: [[media/szz-28/media/image4.png]]

Zdroje

- SZZ okruh 28 — studijní materiály FIT BUT (szz-28.docx). Obrázky: media/szz-28/.

Navigace: [\[\[index|• Přehled okruhů\]\]](#) · [\[\[okruhy/27-strojove-uceni|27. Strojové učení\]\]](#) · další: [\[\[okruhy/29-datove-ridici-struktury|29. Datové a řídicí struktury\]\]](#) [▢](#)